

CODEHIVE

JAVA PROGRAMMING: ITERATION & CONTROL LOOPS

► What are Loops?

Loops repeat a block of code efficiently. They eliminate redundancy and are the foundation of handling repetitive tasks in software.

Analogy: "Repeat the chorus until the song ends."

► The While Loop

Checks the condition **before** executing the code block. If the condition is false initially, the code never runs.

```
int i = 0;
while (i < 5) {
    System.out.println(i);
    i++; // Increment to avoid infinite loop
}
```

► Danger: Infinite Loops

If the loop condition never becomes false (e.g., forgetting `i++`), the program will freeze or crash.

```
while (true) {
    // This code runs forever
}
```

► The Do/While Loop

Executes the code block **at least once** before checking the condition at the end.

```
int j = 10;
do {
    System.out.println("Running at least once");
    j++;
} while (j < 5);
```

► Comparison Summary

Feature	While	Do-While
Check Timing	Pre-execution	Post-execution
Minimum Runs	0	1

► Loop Control: Break & Continue

- **Break:** Terminates the loop immediately.
- **Continue:** Skips the current iteration and jumps to the next check.

```
if (i == 3) continue; // Skip the rest of loop body for i=3
```

► Iteration Summary

- **While:** Ideal for unknown counts (e.g., reading a file until the end).
- **Do-While:** Ideal when you need a guaranteed initial execution (e.g., login menus).
- **Iterator:** Use `i`, `j`, or `k` as standard counter names.
- **Semicolon:** Required after the condition in a do-while loop.

